

Enterprise Control Plane for AI Assisted Product and Software Delivery

Framework choice shared memory and controlled agent improvement

Research white paper | 1 October 2026 | Proposed reference architecture

Recommendation. Establish a small enterprise control plane that governs consequential actions and the evidence required to authorize them. Teams retain their preferred development frameworks. A separately governed memory service helps agents reuse experience, while an independent improvement pipeline determines which lessons, skills and model changes are safe and useful enough to adopt.

The platform covers the full path from a product idea to a measured customer outcome: research, prioritization, requirements, design, test planning, implementation, integration, build, evaluation, release, operation and retirement. It also governs the agents that perform this work. A successful software deployment and an improved agent are different claims; each needs its own evidence.

The central design decisions

Decision	Practical consequence
Keep framework choice open	Accept native artifacts and expose common services. Require trusted evidence at merge, access and release boundaries.
Separate memory from authority	A remembered lesson may inform a proposal. It cannot grant permission, revise policy or certify a release.
Evaluate every learning change	Preserve source evidence, compare against a baseline, release to a limited cohort and retain a working recovery path.
Prove value before broad rollout	Begin with two frameworks and one complete delivery path. Expand only when control effectiveness and net benefit are demonstrated.

This is a design proposal, not a claim that a complete regulated enterprise already operates autonomously. Current research supports specific components and exposes important failure modes. The paper identifies those limits and defines experiments that can change the recommendation.

How to read this paper

This paper is for product leaders, engineering leaders, architects, security teams and platform engineers. It assumes no familiarity with the preceding conversation. Product and engineering examples are hypothetical. Nothing here describes an organization's approved internal architecture or asserts regulatory compliance.

Reading path	Pages
Rationale and research	3-4
Architecture and product lifecycle	5-7
Contracts, framework integration and enforcement	8-10
Execution, evaluation and operational readiness	11-13
Shared memory and Hindsight integration	14-16
Controlled agent improvement and measurement	17-18
Scale, evolution and portability	19-20
Worked example and human operating model	21-22
Disconfirmation and phased delivery	23-24
Technical contract, stop protocol and glossary	25-26
Sources and evidence limits	27-28

Terms used throughout

An **AI agent** is software that uses a model to choose actions, call tools and respond to results. A **framework** supplies a working method, such as specification templates, role instructions or implementation steps. A **harness** is the software that actually runs the agent and its tools. These are distinct from the enterprise services that authorize access and release.

A **control plane** records intended state, evaluates policy and authorizes actions. An **execution plane** performs work in repositories, build systems and environments. **Evidence** is an attributable observation tied to a particular subject, such as a test result for an artifact. **Memory** is information selected for future reuse; it can contain errors or outdated conclusions.

A **dark software factory** means extensive non-interactive execution of software work. In this paper, autonomy is bounded by task, identity, environment and risk. Human accountability remains explicit. The objective is better product outcomes and accepted changes, not a target percentage of human-free work.

First principles and explicit assumptions

The enterprise buys a customer or business outcome. Code, documents, agent activity and pipeline runs are intermediate products. Optimize the time and total cost of obtaining an accepted outcome while meeting constraints on harm, security, reliability and human effort. More output can reduce value if it creates review queues, production defects or unnecessary features.

First principle	Design implication
Intent is incomplete until tested against reality	Record assumptions and unresolved questions. Permit clarification, experiments and a decision to stop.
A claim is weaker than an observation	Check actual application state and behavior. Do not treat an agent's completion statement as a verdict.
Authority exists where an action takes effect	Enforce at repositories, credential services, network gateways and deployment targets.
Learning changes future behavior	Version memory selection, prompts, skills and models as part of an agent release.
Shared components multiply both value and failure	Partition tenants and releases. Limit propagation of new lessons and preserve recovery choices.

Assumptions that must be validated

The design assumes that the enterprise can identify workload owners, protect release credentials, authenticate evidence producers and control production admission. If developers or agents retain an unrestricted alternate release path, the platform can observe that path but cannot honestly claim to govern it.

It also assumes access to suitable test environments and domain experts who can define acceptable behavior. Where those are unavailable, record the gap and keep the work in an advisory or supervised lane. Do not substitute fabricated test evidence or a model's confidence score.

Not every requirement is executable. Accessibility, usability, legal interpretation and strategic value may require specialist judgment or a real-world experiment. Record the responsible evaluator and review criteria. Automation should reduce routine work while keeping consequential judgments explicit.

Assumption record: statement, supporting and opposing evidence, owner, confidence basis, validation action, expiry or review date, and affected decisions. Unknown is a valid state. A missing fact must not silently become an approval.

What the research supports and does not

The evidence supports a modular experiment, not an enterprise-wide autonomy mandate. These sources have different evidentiary strength: documentation describes features; practitioner accounts describe one setting; controlled studies and benchmarks answer narrower experimental questions.

Evidence	Finding and limit	Design consequence
StrongDM factory account February 2026 [1]	Reports agents working from specifications and external scenarios with simulated dependencies. It is a practitioner account, not independent proof of enterprise-wide reliability.	Use protected scenarios; validate simulation fidelity.
Hindsight paper December 2025 [2]	Reports gains on conversational memory benchmarks. These do not establish gains in software delivery, isolation or business outcomes.	Pilot memory on enterprise tasks.
Memory consolidation study May 2026 [3]	Finds repeated textual consolidation can reduce utility. Its experiments and repeat counts limit generalization.	Preserve episodes; test lessons before adoption.
AgentPoison 2024 [4]	Shows retrieved memory can influence agents maliciously in studied settings. It does not give a universal attack rate.	Treat memory as an attack surface.
Anthropic evaluations January and February 2026 [5,6]	Explains outcome grading and shows infrastructure settings can materially alter agent scores.	Evaluate the whole configured system.
METR studies July 2025 and February 2026 [7,8]	An early trial found slowdown; later estimates suggested improvement but selection effects prevented a reliable general estimate.	Measure current local net benefit.
NIST identity project September 2026 [9]	Announces SDLC agent identity and authorization implementation work. This is evolving guidance and demonstration work.	Use explicit identity; do not claim certification.

DORA's 2025 synthesis emphasizes that AI amplifies the surrounding delivery system [10]. The inference for this design is that faster generation pays off only when requirements, verification, release and operating capacity can absorb it. The enterprise must measure that inference in its own setting.

Reference architecture and responsibility

The control plane is a shared decision service with distributed execution. Existing product tools, source control, continuous integration and deployment platforms remain useful. A product registry, evidence index and lifecycle API connect them without replacing every tool.

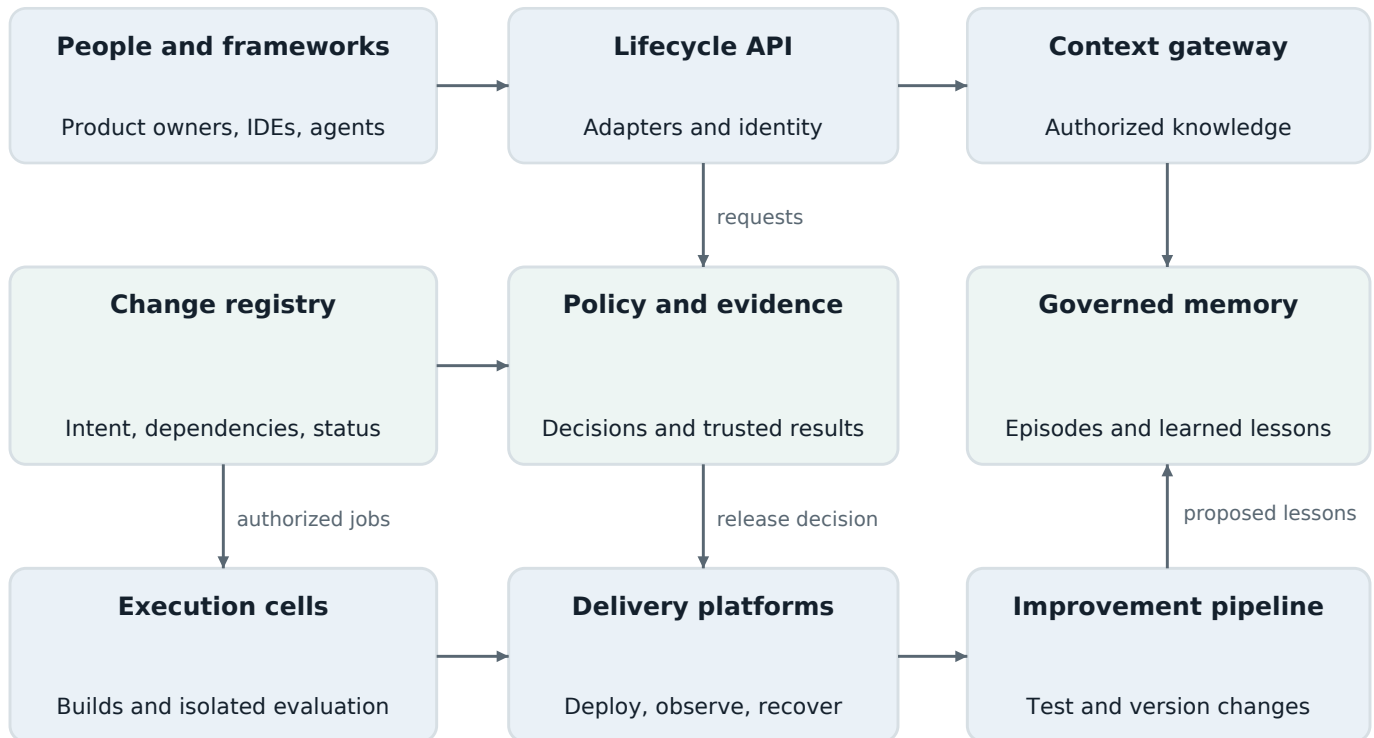


Figure 1. Logical responsibilities and primary request paths. Execution reports and production observations return through authenticated lifecycle events. Memory informs work through the context gateway; it does not authorize actions.

The **change registry** maintains intent versions, owners, dependency relationships and obligations. The **policy service** evaluates a requested action against evidence and current authority. The **evidence store** retains results from approved producers. The **context gateway** supplies current, authorized information and records what each run received.

The **memory service** stores reusable experience behind access and quality controls. The **improvement pipeline** evaluates proposed changes to memory, models and agent behavior. **Execution cells** isolate jobs by tenant, region and risk. A cell is a separately operated unit whose failure should not stop all other units.

These are logical components. Start with a small deployable core and use existing identity, policy, storage and workflow services. OPA can provide policy decisions; Temporal is a possible durable workflow implementation [16,21]. Neither removes the need to define trustworthy inputs and enforce decisions at the destination.

Product discovery through implementation planning

A lifecycle graph records prerequisites and decisions. It is not a mandatory sequence of document approvals. A small repair can inherit valid product context; a new product needs more discovery. Work can run concurrently, and later evidence can reopen an earlier decision.

Activity	Output and accountable judgment	Exit condition
Discovery and ideation	Product owner records the user problem, observed evidence, alternatives and affected users. Research data remains within its permitted purpose.	Problem has evidence or a bounded discovery experiment.
Validation and prioritization	Product owner defines a hypothesis, baseline, expected benefit, disconfirming signal and investment limit. Includes a non-software option.	Proceed, revise or stop decision has an owner and rationale.
Requirements and intent	Domain owner records observable behaviors, negative cases, scope, non-goals and unresolved assumptions. Each requirement has an acceptance method.	Material ambiguities are resolved or bounded by an explicit experiment.
Design and architecture	Engineering owner records interfaces, data flows, dependency choices, threat analysis and migration or compatibility constraints.	Applicable design obligations pass or have authorized exceptions.
Experience and accessibility	Design owner validates user journeys and accessibility criteria with appropriate users or specialists.	Critical journey and accessibility risks have evidence.
Test and delivery planning	Engineering and evaluation owners select checks, test data, dependency environments, observability and recovery actions.	Work can start with scoped permissions and a credible validation path.

A product hypothesis should name an outcome and an observation window. For example: make notification preferences easier to find, then measure completion and support-contact rates without weakening privacy. The initial baseline is measured, not invented. If no baseline exists, instrumentation becomes part of the plan.

Important boundary: product experiments may authorize limited exposure, spending or data collection before a full feature is built. Those actions need their own permissions and evidence; ideation is not automatically consequence-free.

Implementation through operation and retirement

The same change record follows the work across frameworks and tools. The platform recomputes applicable obligations as scope, data sensitivity or dependencies change. It preserves the reason for inherited evidence and invalidates that evidence when its assumptions no longer hold.

Activity	Evidence produced	Required control
Test driven implementation	Visible tests, code, requirement links and relevant failing-then-passing runs.	Protect acceptance criteria. Test-first proof applies where required and meaningful.
Integration and review	Merged candidate, review findings and compatibility checks.	Repository rules and trusted check identities; restricted changes have independent approval.
Build and packaging	Immutable artifact, dependency inventory, source and build provenance.	Approved builder; protected signing; known inputs.
Independent verification	Behavior, security, performance and resilience results for the packaged candidate.	Separate runner identity and protected evaluation assets.
Operational readiness	Owner, SLOs, alerts, runbooks, support model, recovery and migration rehearsals.	Demonstrated readiness appropriate to risk.
Deployment and release	Deployment observations, feature exposure and promotion or recovery decisions.	Authorize exact artifact, configuration and target; verify at execution.
Operation and learning	Reliability, incidents, customer outcomes and validated lessons.	Separate health from value; respond to drift and investigate causes.
Retirement	Dependency removal, traffic drain, credential revocation and data disposition evidence.	Confirm consumers and retention obligations before shutdown.

Track three distinct dimensions: **work state** (proposed, executing, blocked or complete), **assurance state** (unknown, failed, sufficient or expired) and **outcome state** (unmeasured, positive, neutral or negative). A deployed feature can be technically healthy and commercially unsuccessful. A blocked release can be a successful control.

A risk lane is scoped permission, not a permanent label on a team. Authentication, payments, sensitive data or irreversible migrations can require deeper review even when the code change is small.

The common change contract and evidence graph

Use a minimal common contract that references framework-native artifacts. Avoid translating every document into a universal process language. The shared model needs enough structure to answer who wants what, which obligations apply, what was evaluated and which action is currently permitted.

Record	Minimum content
Product outcome	Owner, user problem, hypothesis, baseline status, success measure, observation window and stop condition.
Intent version	Scope, non-goals, requirement identifiers, prohibited effects, assumptions and authoritative approval.
Obligation	Requirement or policy identifier, applicability, evaluator, expected subject, freshness and pass rule.
Candidate	Source commits, artifact digest, configuration, dependency and migration references.
Agent release and run	Model, harness, prompt and skill versions, tools, permissions, memory view, budget and execution environment.
Evidence	Producer identity, subject digest, test and environment versions, result, timestamp, provenance and limitations.
Decision	Requested action, satisfied and unmet obligations, policy version, approver, expiry and revocation epoch.
Observation	Production health or business measurement, population, time interval, data quality and attribution limits.

Relationships make evidence reusable and invalidatable

A requirement is evaluated by a scenario. A result refers to that scenario and a candidate. A decision consumes those results. A release consumes the decision. A product observation refers to the released population. These links form the evidence graph. A relational database with explicit links can implement it; a graph database is optional.

When a requirement changes, invalidate affected scenarios, results and authorizations. When the artifact changes, candidate-specific results are stale. Some context may remain valid, such as an unchanged product owner. Reuse requires declared applicability and dependency rules. If the impact is unknown, rerun the relevant checks or escalate.

A signature demonstrates attribution and integrity within a trust model. It does not prove the test was sufficient, the producer uncompromised or the business outcome achieved. SLSA verification binds provenance to an artifact and trusted builder expectations [18].

Plugging in different development frameworks

Offer three integration modes. **Artifact mode** imports specifications and changes produced elsewhere. **Interactive mode** exposes context, obligations and evaluation requests in developer tools. **Managed mode** runs a pinned framework inside a controlled execution environment. A framework can support one or several modes.

Framework example	Useful native contribution	Proposed enterprise mapping
GitHub Spec Kit [11]	Structured specifications, plans, tasks and verification workflows.	Map requirement and plan references to the common intent record.
OpenSpec [12]	Change proposals, specifications, design and tasks.	Preserve change artifacts and map their scenarios to obligations.
BMAD with TEA [13,14]	Product and architecture workflows plus test planning, traceability and CI review.	Import links and results; verify producer, candidate and evaluator authority.
Kiro [15]	Structured feature or bug-fix specifications and implementation tasks.	Use approved export or integration surfaces; validate version-specific capabilities.

These are adapter designs, not claims that the named projects already implement this control-plane contract. Other frameworks can participate through the same modes. An enterprise can initially allow artifact contribution without approving a framework to execute sensitive workloads.

The adapter contract

Declare the adapter and framework version, supported activities, artifact schemas, requirement mapping rules, permitted tools, requested scopes and lifecycle events. Managed execution additionally needs start, status, cancel, heartbeat and recovery semantics. Keep unmapped fields visible and reject unsupported required fields; never manufacture a passing obligation from a missing field.

Conformance tests must cover framework upgrades, duplicate events, schema evolution, cancellation, malicious artifacts and a mid-change framework switch. The switch changes the implementation method, not outstanding enterprise obligations. Pin executable packages and review dependency changes.

Artifact mode cannot prove every action taken on an unmanaged laptop. Label that assurance limit and rerun required checks in trusted infrastructure. MCP may expose platform tools, but transport authorization alone does not define lifecycle policy or evidence trust [22].

Enforcement at the point of consequence

Framework instructions describe desired behavior. Enforcement services constrain what can actually happen. The platform should minimize the trusted core: identity and credential issuance, policy evaluation, source and artifact integrity, approved evidence producers and the destination that executes a consequential action.

Boundary	Enforcement mechanism	Failure response
Context and tools	Authenticate caller; derive tenant and resource scope on the server; allow only approved tools and destinations.	Deny unauthorized retrieval or action; record reason.
Source control	Require results from recognized producers for the exact revision; protect policy and acceptance assets.	Reject merge or invalidate stale checks.
Evaluation	Use protected suites and separate runner identity; verify subject and result schema.	Return failed, inconclusive or infrastructure error distinctly.
Credential issuance	Issue task-scoped, short-lived capabilities with an audience, resource and action.	Deny expired, revoked or broader-than-approved requests.
Release target	Validate candidate tuple, decision expiry, policy freshness and revocation status at use time.	Pause promotion and reconcile state.

OPA separates policy evaluation from the integrating service that enforces the result [16]. An HTTP success code is not an allow decision. Missing, malformed or undefined policy results must fail closed for privileged actions. Authoritative identity and resource attributes must come from trusted services, not model-authored payloads.

Release authorization binds intent, source or artifact, configuration, environment, applicable evidence and policy. It names the consuming service and expires. Recheck at use time to prevent a candidate swap after approval. Give each logical action an idempotency key; use an attempt identifier to distinguish retries.

Emergency exceptions name an accountable authority, resource scope, compensating controls and expiry. They are visible in the evidence record and reviewed afterward. The platform cannot guarantee control while unrestricted alternate credentials remain usable; discovering and closing those paths is part of onboarding.

Human approvals should contain a bounded decision and a concise evidence summary. Requiring a person to rubber-stamp every routine agent action creates a queue without demonstrating better control.

Containerized execution and reproducible testing

Separate implementation, trusted build, independent evaluation and production release identities. The framework may change its workspace and visible tests. It cannot edit protected evaluation assets, write the verdict or access production credentials.

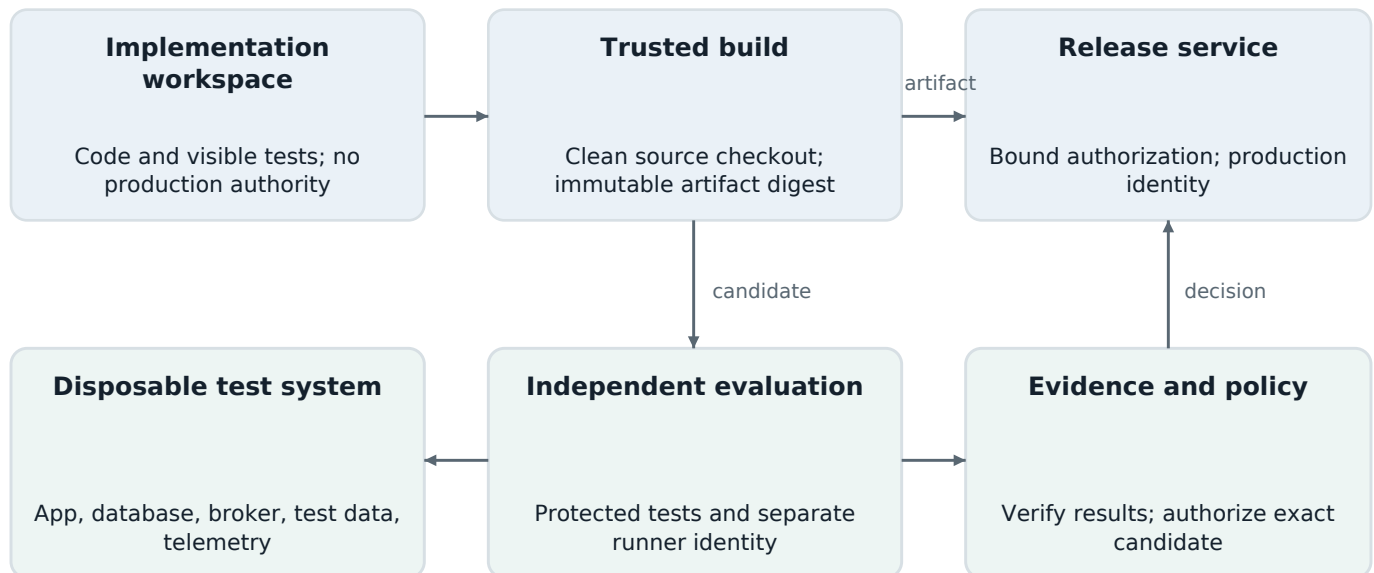


Figure 2. The build exports an immutable candidate. The evaluation service tests that candidate against a disposable system and emits evidence. The release service consumes a verified decision.

A disposable test system can contain the application, database, broker, synthetic data, dependency simulators and telemetry collector. Testcontainers is one way to start dependencies for tests [20]. Use real dependency versions where practical; test simulators against provider contracts and selected real integrations to detect drift.

An **environment contract** pins image digests, operating system and architecture, dependency topology, data seed, locale, clock behavior, CPU reservation, memory ceiling, timeout, cache policy and network rules. Resource settings can alter measured agent performance [6]. Record infrastructure failures separately and include them in operational reliability reporting.

Use non-root execution, restricted mounts, scoped workload identity and network controls. Untrusted generated code may require a hardened sandbox or virtual machine boundary. Kubernetes namespaces require additional isolation controls [19]. A test runner with a host container-engine socket can undermine isolation; place such runners in an appropriately isolated worker environment.

Reproducibility means preserving inputs and making reruns comparable. It does not promise identical model output, scheduling or external service behavior. Retain the actual outputs and tool effects needed to investigate a run.

Evaluation that can reject a plausible answer

Use four evaluation layers: intent quality, permitted execution, software behavior and real-world outcome. An agent can satisfy one and fail another. Missing evidence, an evaluator crash or an unreachable dependency is an inconclusive result, not a pass.

Layer	Example check	Evaluator
Intent	The owner can distinguish success, failure and out-of-scope behavior; ambiguous cases have a resolution.	Domain expert plus consistency checks.
Execution	Tools, destinations, files and spending remained within the delegated task.	Gateway records and deterministic policy.
Candidate	Correct authorization, contracts, performance, migrations and recovery behavior.	Protected test runners and specialist review.
Production and product	The released population is healthy and the intended customer outcome is measured.	Telemetry and product analysis with attribution limits.

Prefer deterministic checks for exact behavior. Use model judges for semantic judgments when necessary, with a rubric and human calibration. Inspect the resulting system state. Repeated trials reveal reliability that a single successful run misses [5]. A separate evaluator agent is not independent if it shares writable tests, credentials or authority with the implementer.

Protect the evaluator itself

Maintain known-good and deliberately faulty candidates to validate graders. Use mutation tests or seeded defects to see whether a suite detects meaningful failures. Test model judges for prompt injection, style bias and disagreement. Distinct models can reduce some correlated errors, but model diversity does not establish independence by itself.

Keep protected acceptance cases outside the implementing agent's write and read scope. Supply visible examples for development and bounded failure feedback for repair. Repeated probing can reveal hidden tests; cap retries, rotate scenarios and exclude protected test content from shared memory. An acceptance scenario can later be published as a regression test through an intentional process.

Where test-driven development is required, retain a trusted failure against the old behavior and a pass against the candidate using the intended assertion. A compilation error or timeout is not proof of a meaningful red test. Test-first sequencing is not equally useful for every discovery, design or documentation task.

Release readiness and production outcomes

A release requires both a technically acceptable candidate and a credible operating plan. Name the service owner and on-call route. Define service-level objectives, dependencies, dashboards, alert actions, support documentation, capacity constraints, data migration controls and an exercised recovery procedure.

Promote the evaluated candidate

Deploy by immutable artifact digest with separately versioned configuration, feature flags and migration references. Verify the authorization at the destination. Record what was actually deployed and exposed to users. Avoid rebuilding between evaluation and promotion unless the resulting artifact is independently verified again.

Use progressive exposure with a baseline or control population where practical. Define metrics, minimum traffic and observation windows before the rollout. Low traffic or conflicting signals should produce an inconclusive decision, not an automatic pass. Account for delayed harms and correlated incidents; a brief healthy canary cannot establish long-term business value.

Recovery is part of the design

Automated recovery may disable a flag, route traffic back, restore a compatible version or execute a compensating action. A binary rollback cannot undo an email, financial transfer or destructive database migration. Rehearse data compatibility and forward-repair options before granting autonomous release authority.

Observation	Meaning	Next action
Healthy system, weak adoption	The feature may work technically but fail its product hypothesis.	Investigate usability, relevance and measurement before adding more code.
Healthy average, harmed subgroup	Aggregate metrics conceal an unacceptable effect.	Limit exposure and inspect cohort-specific outcomes.
Reliability regression	The candidate or its environment may be unsafe.	Execute the approved recovery policy and retain evidence.
Incident repeated	A learned lesson or regression check failed to transfer.	Investigate applicability, retrieval and evaluator coverage.

Post-release observations are inputs to investigation. They do not automatically prove causation or become enterprise-wide rules. Product decisions and agent improvement proposals follow separate review paths.

Shared memory as governed reusable experience

Shared memory should mean authorized reuse across an explicit scope. It should not mean one global store every agent can read and rewrite. Separate the authoritative source archive, the memory derived from it and the policy that governs action.

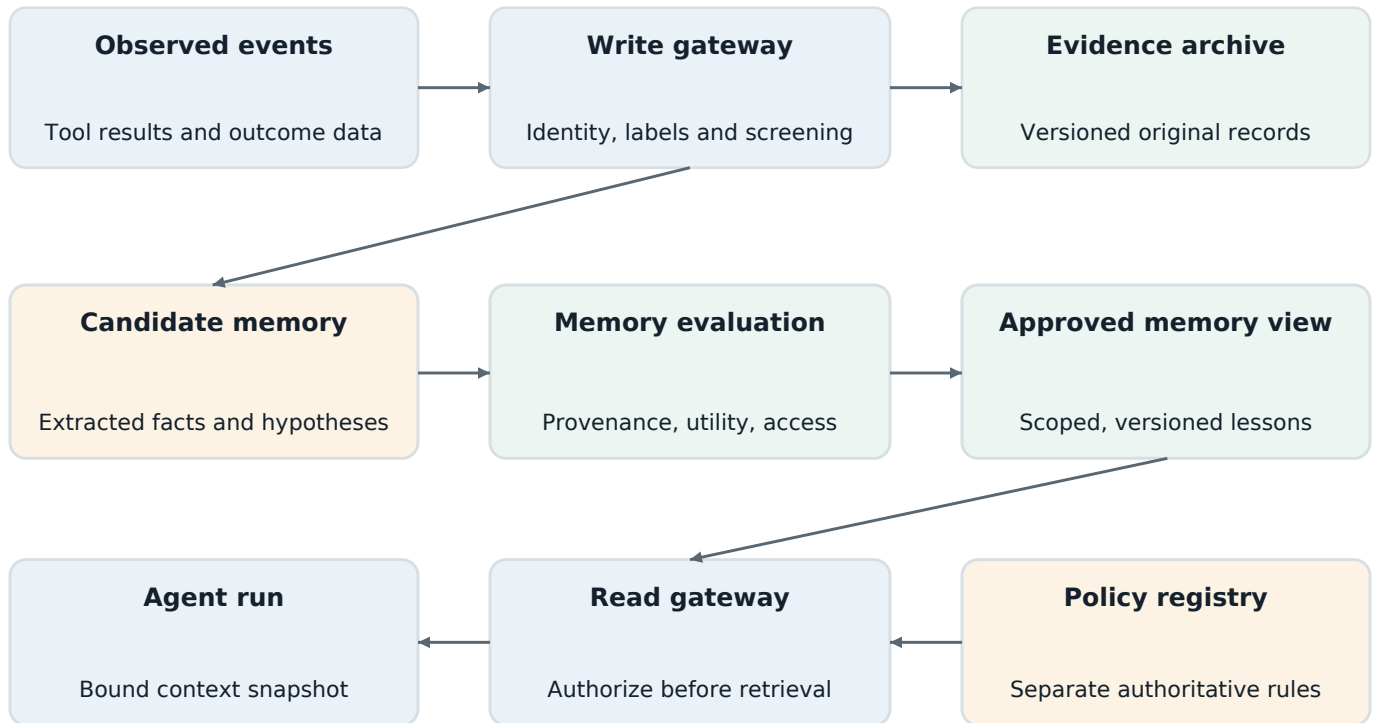


Figure 3. Proposed memory boundary. Derived lessons remain candidates until evaluated. Read and write gateways enforce scope. The policy registry remains an independent source of authority.

Use five memory classes: **working context** for the current task; **episodes** recording what happened; **domain knowledge** linked to current sources; **procedures** that have been reviewed and tested; and **hypotheses** awaiting validation. Give each class different access, expiry and promotion rules.

Each reusable item carries an owner, tenant, project or domain scope, source identifiers, observed time, valid time, confidence basis, sensitivity, permitted purpose and supersession links. Label observations and inferred conclusions distinctly. A frequently retrieved claim is not necessarily true.

Retain original permitted evidence outside the derived memory index under an appropriate retention policy. Summaries may omit distinctions needed later. Store human-readable decisions and tool results; do not require hidden model reasoning. The memory consolidation study supports testing abstractions rather than assuming they improve with every rewrite [3].

A run records an authorized memory view and the exact retrieved items or a durable retrieval manifest. A mutable bank identifier alone is insufficient to reconstruct what influenced the agent.

Integrating Vectorize Hindsight

Hindsight is a candidate memory engine behind the enterprise memory API. Its documented retain, recall and reflect operations provide extraction, retrieval and synthesis, while banks organize separate memory contexts [23]. The platform should remain usable if this engine is replaced.

Hindsight capability	Proposed platform use	Enterprise responsibility
Retain	Ingest permitted episodes and source references.	Authenticate writer; screen secrets; assign scope; preserve original evidence.
Recall	Retrieve relevant items for a task.	Derive access scope on the server; verify source permissions and freshness before disclosure.
Reflect	Draft a diagnosis or reusable lesson.	Treat the output as a proposal; retain provenance and test its utility.
Banks and tags	Organize tenant, project and domain views.	Enforce bank authorization; apply access restrictions to every retrieval and synthesis path.
Mental models and directives	Provide selected summaries and reflection instructions.	Version their use; restrict modification; keep policy enforcement outside prompts.
Extensions	Integrate authentication, tenant resolution and operation checks.	Implement and test the identity and authorization mapping for the chosen deployment.

Documentation distinguishes tenant extensions from operation-validation hooks. The simple built-in API-key extension uses the public database schema for authenticated requests; tenant-specific isolation requires an appropriate implementation [24]. Bank naming and a successful login do not by themselves authorize a caller to access every bank.

The documentation describes directives as instructions used during reflection [25]. Regardless of their wording, enforce enterprise permissions and release rules in deterministic services. The framework must not receive unrestricted memory administration tools such as clearing banks or editing approved instructions.

Pin an exact engine version and configuration in the pilot. Test all read, reflect, export and administration paths. The documentation describes bank transfer and re-embedding on import [25]; benchmark retrieval after migration because an exported archive does not guarantee identical rankings or behavior.

The wrapper, immutable run manifests, approval states and learning promotion controls described here are proposed enterprise components. They are not asserted to be built-in Hindsight guarantees.

Memory permissions freshness and poisoning

Use the same source access rules for derived memories as for the original information. Authorize before candidate retrieval and before any synthesis. Post-filtering an answer is too late if restricted information already influenced it. Derived summaries inherit the combined restrictions of their inputs unless an authorized declassification process applies.

A complete write and read path

A write gateway authenticates the producer, checks purpose and scope, screens secrets and suspicious instructions, and records the source. New content enters a candidate store. A read gateway derives the permitted banks and filters from trusted identity, excludes revoked or expired items, and verifies applicability to the current version and environment.

Hindsight documentation warns that permissive tag modes can include untagged content [23]. Use explicit, tested partitions; do not let an agent choose its own tenant filter. Keep approved cross-team lessons in a separate published view with their own owner and provenance.

Failure mode	Required response
Poisoned tool output becomes a lesson	Quarantine it; trace downstream use; assess affected runs and retract derived items.
A source is deleted or access is revoked	Invalidate derived facts, summaries, caches and exported views; verify the deletion or restriction workflow.
Two sources contradict each other	Preserve both with scope and time; prefer current authoritative evidence; escalate unresolved conflict.
A lesson applies to an older platform	Require version applicability; do not rank relevance above validity.
Protected tests leak into memory	Stop further distribution, rotate compromised evaluations and rebuild clean learning views.

Memory poisoning research demonstrates that ordinary-looking retrieved material can redirect agents [4]. Content scanning helps but cannot guarantee safety. Combine narrow capabilities, source attribution, independent checks and adversarial tests. Repeated paraphrasing must not wash away the original trust label.

Apply retention, deletion and legal-hold requirements to the evidence archive and memory separately. Record deletion tombstones so a restored backup cannot silently republish removed content. Sensitive data must remain protected in embeddings, graph edges, telemetry and caches as well as visible text.

Improving agents through controlled iterations

Agent improvement is a release process. The changed unit may be a skill, prompt, tool, retrieval policy, memory lesson, model or harness. Persistent memory usually changes supplied context; it does not automatically change model weights. Fine-tuning is a separate, explicitly governed option.

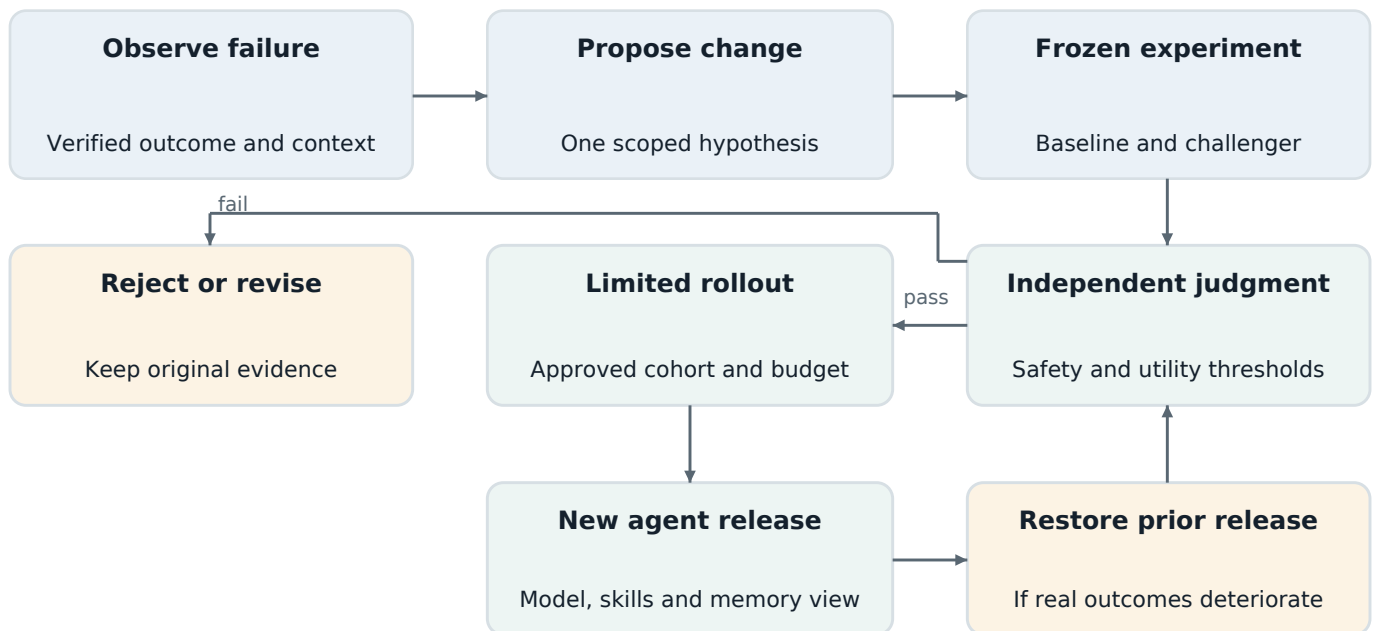


Figure 4. Improvement cycle. Production observations initiate a hypothesis. Independent tests and a limited rollout determine adoption. Failed changes preserve the current release and the original evidence.

First establish a reproducible baseline and identify a specific failure class. Propose one change with an expected benefit and a plausible downside. Freeze candidate inputs, evaluator versions, environment and learning state. Compare the baseline and challenger on unseen tasks, adversarial cases and existing regression suites.

An independent authority approves promotion within a risk scope. Publish an agent release manifest containing the model, harness, skills, tools, permissions, retrieval configuration and approved memory view. Roll it out to a limited cohort and compare actual outcomes. Restore the previous version when guardrails deteriorate.

Separate **within-task repair** from **cross-task learning** and **platform self-modification**. Repair can remain inside the task budget. Shared lessons require evaluation. Changes to policy, evaluator definitions or privileged platform code require separate ownership and release controls.

Agents may propose improvements to the platform itself. They do not get authority to weaken their own acceptance criteria, expand their permissions or promote themselves. Any recursive improvement uses the same external gates and bounded budgets.

Measuring whether learning actually helps

The core counterfactual is simple: would the same task cohort have performed as well without the proposed change? More memories, higher confidence, more tool calls and agent self-ratings do not answer that question.

A practical experimental design

Compare three conditions: the current agent with current memory, the same agent without reusable memory, and the challenger with the proposed memory change. Keep models, task budgets, environments and evaluators fixed unless that component is the treatment. Randomize task order and use repeated runs. Reset writable learning state between trials to avoid contamination.

Split development and evaluation by repository family and time where possible. Near-duplicate tasks can leak solutions across a nominal holdout split. Protect a final evaluation set from optimization feedback. Use paired analysis by task and account for correlated tasks; count cancelled, timed-out and infrastructure-failed runs explicitly.

Measure	Operational definition
Accepted-change lead time	Approved intent to validated release; also report discovery time separately.
Human effort	Hands-on clarification, review, repair and incident effort per accepted change.
Total cost	Inference, environments, evaluation, storage and human work, including failed attempts.
Reliability	Per-trial task success plus all-trials-success for a declared repetition count.
Memory benefit	Difference in accepted outcomes and cost versus the frozen current and no-memory conditions.
Safety and access	Unauthorized effects, cross-scope disclosure, stale-source use and failed containment tests.
Product value	Predefined user or business outcome over a declared observation period.

Define success margins and analysis before the experiment. Promote only if benefit is credible and guardrails remain within agreed limits. A small trial with no observed harm does not establish a very low failure rate. Under an independent-trial approximation, zero failures in 300 trials still permits an upper 95% failure-rate bound of roughly 1%; correlated trials provide less information.

All-trials-success is empirically measured. Writing it as p to the power k assumes independent trials with constant success probability; real agent runs often violate that assumption. Report the task mix and uncertainty, not only one aggregate score.

Scale reliability and interruption

Separate the fast authorization path from asynchronous indexing, analytics and learning. A production admission decision must not depend on a long reflection call. Use tenant and regional execution cells with quotas, independent queues and failure containment. Shard by stable ownership keys and avoid a single global transaction for ordinary work.

Use transactional updates for a change's authoritative state, with an outbox for events and idempotent consumers. Reconcile the recorded state with repositories, registries and deployment targets. A durable workflow can resume after a crash, but it cannot make an external side effect occur exactly once without cooperation from the destination.

Failure	Expected behavior
Memory unavailable	Continue with an approved no-memory mode if allowed; otherwise pause the affected task. Never relax policy.
Policy or evidence unavailable	Pause new privileged actions unless a pre-authorized, fresh and narrowly scoped fallback applies.
Duplicate or delayed event	Deduplicate; verify subject version and sequence; reconcile actual target state.
Cell outage	Limit the blast radius; resume from durable state after checking external effects.
Regional disconnection	Respect data residency and authorization freshness; do not migrate sensitive state opportunistically.

Emergency stop with an honest guarantee

Support task, agent-release, tenant and enterprise stops. Block new authorizations; revoke capabilities; fence stale workers; interrupt managed jobs; then reconcile committed effects. Destinations should reject stale authorization epochs. Measure stop propagation under network partitions and define a maximum allowed action window.

Do not promise instantaneous revocation of all offline credentials. Bound token and lease lifetimes and require freshness checks for consequential actions. Keep a separately authorized recovery route so stopping new changes does not prevent operators from mitigating harm. Running business services should continue when the development control plane is unavailable.

OPA policy bundles can be distributed with eventual consistency [17]. The platform must add explicit freshness, revocation and fallback rules appropriate to each action. Eventual distribution is not an immediate enterprise-wide stop mechanism.

Evolving the platform without losing control

Future models, frameworks, agent protocols and memory engines should be replaceable behind stable enterprise contracts. Standardize the meaning of intent, authority, evidence and outcomes. Let execution details evolve through versioned adapters and conformance tests.

Change	Safe evolution mechanism
New framework or IDE	Begin in artifact mode; run conformance and threat tests before enabling managed privileges.
New model or harness	Create a new agent release; compare with the existing release on frozen tasks and monitor limited deployment.
New memory engine or embeddings	Export governed source records; reconstruct derived indexes; rerun access, retrieval, deletion and utility tests.
New policy or evaluator	Use shadow decisions first; inspect changed outcomes; obtain independent approval; invalidate affected authorizations.
Schema version change	Support explicit versions and compatibility rules; reject unknown required fields; migrate without discarding obligations.
Jenkins or Spinnaker to GitLab	Preserve candidate, evidence and decision semantics; translate execution steps through a tested adapter.

Keep a capability registry that declares supported schema versions, operations and limits. Clients negotiate capabilities and receive a clear unsupported response. Avoid embedding a framework's internal role names into the enterprise contract.

A release manifest should identify all material behavior dependencies, including prompts, tools, models, retrieval configuration and the approved memory view. Some hosted models and external systems cannot be pinned indefinitely. Record their observable identity, rerun sentinel evaluations and restrict autonomy when an unreviewed change invalidates confidence.

For runtime agents that keep learning, use a declared memory epoch or read snapshot for each evaluation and release cohort. New writes can accumulate in a candidate view until promotion. If freshness requires live facts, keep those in an authorized current-source channel and record exactly what was retrieved.

Portability has a test: replace one adapter or memory engine without changing the meaning of an existing release decision. If the replacement silently changes authority, evidence eligibility or scope, the abstraction has failed.

Worked example from idea to a better agent

Consider a hypothetical notification-preferences service. A product owner wants users to find and update their preferences more easily. The team uses BMAD for discovery and OpenSpec for implementation. Jenkins builds the service and Spinnaker releases it. The same design can use other tools.

Step	What happens and what is recorded
1 Product intent	The owner records the problem, baseline measurement plan, success window and prohibited cross-customer changes. No adoption gain is assumed.
2 Requirements	REQ-01 allows users to edit their preferences. REQ-02 forbids editing another user's preferences. Each has an evaluator and scenario links.
3 Framework switch	The OpenSpec adapter imports BMAD references. Unresolved privacy and recovery obligations remain on the same change record.
4 Implementation	The agent receives scoped context, visible tests and a workspace. A proposed memory lesson about caching is marked applicable only to an older service version.
5 Independent failure	A trusted test attempts cross-customer editing against candidate A and detects an authorization defect. The result blocks release.
6 Repair and release	The team fixes the defect. Candidate B passes required checks. A decision bound to B and its configuration authorizes a limited rollout.
7 Observe	Operations checks health; product analysis separately measures preference completion and support contacts. An inconclusive outcome remains open.
8 Learn	The failure becomes a permitted episode. A proposed lesson about authorization test coverage is evaluated on other unseen service changes before publication.

The reusable lesson is scoped: include cross-owner negative tests for this API pattern. It does not say that every service should use the same authorization implementation. A learning experiment checks whether the lesson prevents similar defects without adding incorrect restrictions elsewhere.

If the challenger agent performs worse, keep the episode but reject the generalized lesson. If it improves, publish a new scoped memory view and agent release. The platform can restore the previous release without deleting the incident evidence. Framework preference never determines whether REQ-02 is enforced.

The human operating model and user experience

The platform should make the next useful action easy to understand. Developers need to see what is missing, why it matters and how to resolve it. Product owners need a view of assumptions and outcomes. Control owners need evidence of effectiveness and exceptions, not a count of uploaded documents.

Role	Accountable decision	Useful view
Product owner	Problem, intended outcome, prioritization and experiment decisions.	Hypotheses, user evidence, value signals and unresolved assumptions.
Engineering owner	Design, candidate quality and readiness to operate.	Dependencies, failed scenarios, remediation and release state.
Evaluation owner	Scenario quality, grading validity and protected test management.	Coverage, seeded-defect detection, false passes and judge calibration.
Policy owner	Control rules, exceptions and approval boundaries.	Changed decisions, bypass attempts, expiry and control failures.
Memory steward	Published lesson scope, source quality and withdrawal.	Provenance, contradictions, access changes and measured utility.
Platform and operations owners	Availability, isolation, credentials and recovery capability.	Queues, costs, stop propagation, service health and reconciliation.

These are accountabilities, not a requirement for six new departments. Smaller organizations can combine roles when separation of duties is preserved for sensitive decisions. The implementer must not be the only authority able to change acceptance criteria and declare success.

A practical change screen

Show the intent and owner, current candidate, applicable obligations, three assurance states (passed, failed, missing), and the next permitted action. A blocked row should link to the exact failing scenario and responsible service. Approval should display scope, evidence, limitations and expiry. Include an accessible interrupt action for authorized users.

An internal engineering portal can host these views while IDEs and command-line tools use the same APIs. Keep notifications actionable and batched. A platform that makes simple work slower will encourage bypass. Track time spent resolving platform errors separately from time spent improving the product.

Tests that could disprove the design

A credible platform needs falsifiable claims. The following tests should run before expanding autonomy. Passing is evidence within the test scope; it is not proof that every future failure is impossible.

Claim under test	Attempt to disprove it	Failure changes the recommendation
Framework neutrality	Switch frameworks halfway through a change with an unmet control.	Fix the contract if the obligation vanishes or semantics change.
Trusted release	Reuse a passing result for a changed artifact, configuration or target.	Do not expand release authority until rejection is verified.
Complete mediation	Attempt release using an old credential or alternate API.	Keep the path supervised until the bypass is closed.
Evaluator integrity	Submit a known-bad candidate and misleading agent completion text.	Repair graders and re-evaluate affected releases.
Isolation	Request another tenant's bank, export, summary or graph-linked facts.	Halt affected memory sharing and fix authorization.
Memory safety	Plant a plausible instruction in a permitted tool result.	Reduce capabilities and quarantine derivative memories.
Memory utility	Compare current, no-memory and challenger conditions on unseen work.	Keep simpler retrieval if memory adds cost or regressions.
Source withdrawal	Revoke a source then query summaries, caches and restored backups.	Do not onboard sensitive data until withdrawal is complete.
Stop effectiveness	Interrupt during network loss and after credential issuance.	Shorten leases or require stronger destination checks.
Durable execution	Crash after an external action but before recording completion.	Require destination idempotency or supervised reconciliation.
Net productivity	Measure accepted-change time, human effort and escaped defects.	Reduce scope or stop if gains do not exceed operating cost.
Future portability	Replace a framework or memory provider with a conformant alternative.	Revise interfaces if policy meaning or evidence is lost.

Define test owners and acceptance thresholds before implementation. Retain failures as regression cases. A failed assumption should change the architecture or rollout decision, not merely produce a new document explaining it away.

Phased adoption and investment decisions

The sequence below is illustrative, not a delivery commitment. Actual duration depends on existing identity, repository controls, test environments and integration capacity. Advance by demonstrated exit conditions rather than calendar dates.

Phase and indicative horizon	Deliverable	Exit evidence
1 Observe Weeks 1-4	One product cohort; minimal change contract; baseline metrics; authoritative identities and evidence links.	Missing evidence and bypass paths are visible; no invented passes.
2 Enforce Weeks 5-8	Two frameworks, one trusted build and evaluation path, one release target and scoped authorization.	Candidate swaps, spoofed checks and alternate credentials are rejected.
3 Learn carefully Weeks 9-12	Governed episodes, Hindsight behind a wrapper, no-memory baseline and one evaluated lesson.	Cross-scope access fails; deletion works; improvement has measured utility.
4 Expand selectively Months 4-6	Additional cohorts, self-service cells, product outcome feedback and adapter upgrades.	Benefits persist; operations and controls meet agreed service objectives.

Start with the smallest complete path

Choose a reversible service change with good testability and a willing product owner. Include enough complexity to test real controls, but avoid making an irreversible financial or identity change the first autonomy pilot. A framework comparison is useful only when tasks and acceptance standards are comparable.

Suggested pilot service objectives are explicit decisions to calibrate: for example, a 250 ms p95 budget for local authorization, less than 30 seconds to block new privileged actions after a connected stop, and a five-minute maximum capability lifetime where permitted. These are proposed targets, not measured product capabilities or universal safe values. Higher-risk actions may require immediate online checks.

Before scaling, agree on the minimum meaningful lead-time or effort improvement, tolerated cost, required defect controls and observation period. Fund evaluation ownership, memory stewardship and incident support as part of platform cost. If the benefit depends on unrecorded expert cleanup or unsafe bypasses, the pilot has not succeeded.

The initial product is a trusted path from intent to one validated outcome. A large framework catalog, global memory lake or fully autonomous platform is not a prerequisite.

Appendix A A concrete platform contract

The following is illustrative platform syntax, not a native Hindsight or framework API. Content references resolve to versioned enterprise records. Identity and entitlement fields are supplied or validated by trusted services.

```
change:
  id: CHG-1042
  schema_version: 1
  tenant: retail-example
  owner: team-notifications
  intent_ref: intent/1042/v3
  requirements: [REQ-01, REQ-02]
  assumptions_ref: assumptions/1042/v2
candidate:
  source_commit: <commit-sha>
  artifact_digest: sha256:<digest>
  config_digest: sha256:<digest>
  migration_ref: migrations/1042/v1
agent_run:
  release_ref: notification-agent/v7
  framework_ref: adapter/openspec/<pinned-version>
  context_manifest_ref: context/run-81
  memory_view_ref: memory/project-12/epoch-6
  environment_ref: test-env/v4
evidence:
  obligations_ref: obligations/1042/v5
  result_refs: [eval/224, build/119, readiness/36]
release_request:
  action: deploy_canary
  target: service-notifications/prod
  idempotency_key: CHG-1042-candidate-B-canary
  decision_ref: decision/539
```

Operation	Purpose
Submit intent or artifact	Create a new version without erasing earlier evidence.
Query obligations	Return unmet checks, owners and permitted next actions.
Request execution or evaluation	Create a budgeted job under a named identity and environment.
Request consequential action	Evaluate authority against the exact subject and current policy.
Recall or propose memory	Retrieve within authorized scope or submit a candidate lesson.
Interrupt and reconcile	Fence new work, cancel jobs and verify external effects.

Events require tenant, subject version, producer identity, event identifier, occurred time, received time and causal reference. Authenticate producers and verify external facts; do not accept a model-authored event as proof that a deployment occurred.

Appendix B Stop protocol and glossary

Minimum interruption protocol

1. Authenticate the operator and record stop scope and reason. 2. Increment the revocation epoch and reject new authorizations in scope. 3. Notify destinations and credential services; fence stale attempts. 4. Cancel managed executions and block further tool actions. 5. Inspect effects already committed and execute separately authorized recovery. 6. Resume only after an accountable decision and fresh authorization.

Record request time, enforcement time at each destination, last accepted action and unresolved effects. Test the protocol with unavailable workers and network partitions. An unresponsive worker is not proof that it stopped.

Glossary

Term	Meaning in this paper
Adapter	A versioned integration that maps a tool's artifacts and actions to the platform contract.
Capability	A scoped authorization to perform an action on a resource, usually time limited.
Digest	A cryptographic fingerprint used to identify exact content.
Epoch	A monotonically changing version used to distinguish authorized state from stale state.
Holdout	Evaluation cases protected from direct optimization or implementation access.
Idempotency	Repeating a logical request does not repeat its intended external effect.
MCP	Model Context Protocol, an interface for exposing tools and context to AI applications.
Memory consolidation	Turning episodes or facts into broader summaries or lessons.
OPA	Open Policy Agent, a policy decision engine.
SLSA	Supply-chain Levels for Software Artifacts, a framework for software supply-chain assurance.
SLO	Service-level objective, a stated reliability or performance target.
Tenant	An organizational or customer boundary with its own identity, data and permissions.
Trusted producer	A service authorized to make a specific evidence claim within a defined scope.

Sources and evidence notes

Sources accessed 1 October 2026. Numbered citations identify factual inputs. Architecture, control rules, thresholds and rollout choices are recommendations developed in this paper. Product documentation can change; validate a pinned version before implementation.

[1] StrongDM. Software Factories And The Agentic Moment. 6 February 2026. [Open source](#)
Practitioner account; used for scenarios and dependency simulation, not a general enterprise success claim.

[2] Latimer and colleagues. Hindsight is 20/20 Building Agent Memory that Retains Recalls and Reflects. 14 December 2025. [Open source](#)
Research preprint; benchmark results concern conversational memory.

[3] Zhang and colleagues. Useful Memories Become Faulty When Continuously Updated by LLMs. 13 May 2026. [Open source](#)
Research preprint; identifies consolidation regressions with explicit experimental limitations.

[4] Chen and colleagues. AgentPoison Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. 17 July 2024. [Open source](#)
Primary research on poisoning retrieved memory and knowledge.

[5] Anthropic. Demystifying evals for AI agents. 9 January 2026. [Open source](#)
Engineering guidance; outcome grading, repeated trials and calibrated evaluation.

[6] Anthropic. Quantifying infrastructure noise in agentic coding evals. 5 February 2026. [Open source](#)
Controlled engineering study; environment resources affect scores.

[7] METR. Measuring the Impact of Early 2025 AI on Experienced Open Source Developer Productivity. 10 July 2025. [Open source](#)
Randomized study in a narrow developer and task population; not a current universal effect.

[8] METR. We are Changing our Developer Productivity Experiment Design. 24 February 2026. [Open source](#)
Follow-up explains selection bias and uncertainty in newer productivity estimates.

[9] NIST. Comments on Software and Agentic AI Identity Concept Paper. 29 September 2026. [Open source](#)
Active project update; not a certification or completed regulatory standard.

[10] DORA. State of AI assisted Software Development 2025. 2025. [Open source](#)
Research synthesis about the wider delivery system; avoid causal claims beyond the study.

[11] GitHub. Spec Kit repository and documentation. Current documentation. [Open source](#)
Framework features; enterprise adapter mapping is proposed in this paper.

[12] Fission AI. OpenSpec repository and documentation. Current documentation. [Open source](#)
Framework artifacts and workflow; does not certify enterprise control effectiveness.

Sources and evidence notes continued

Sources accessed 1 October 2026. Numbered citations identify factual inputs. Architecture, control rules, thresholds and rollout choices are recommendations developed in this paper. Product documentation can change; validate a pinned version before implementation.

[13] BMAD. BMAD Method repository. Current documentation. [Open source](#)
Product and software workflow framework.

[14] BMAD. Test Architect enterprise module. Current documentation. [Open source](#)
Traceability and CI review capabilities; actual control quality requires local validation.

[15] Kiro. Specs documentation. Current documentation. [Open source](#)
Structured development artifacts; feature availability varies by product surface.

[16] Open Policy Agent. Integrating OPA. Current documentation. [Open source](#)
Policy evaluation interfaces and integrating enforcement responsibilities.

[17] Open Policy Agent. Bundles. Current documentation. [Open source](#)
Policy distribution and eventual consistency.

[18] SLSA. Build Verifying artifacts version 1 point 2. Approved specification. [Open source](#)
Artifact provenance verification and trusted builder expectations.

[19] Kubernetes. Multi tenancy. Current documentation. [Open source](#)
Isolation considerations beyond namespaces.

[20] Testcontainers. Testcontainers project. Current documentation. [Open source](#)
Disposable dependency environments for testing.

[21] Temporal. Temporal Platform Documentation. Current documentation. [Open source](#)
Possible durable workflow implementation; product selection requires assessment.

[22] Model Context Protocol. Authorization specification. 25 November 2025 revision. [Open source](#)
Protocol authorization; not a complete enterprise lifecycle policy.

[23] Vectorize Hindsight. Best Practices. Current documentation. [Open source](#)
Retain recall reflect operations, bank structure and filtering cautions.

[24] Vectorize Hindsight. Extensions. Current documentation. [Open source](#)
Tenant authentication and operation-validation extension responsibilities.

[25] Vectorize Hindsight. Memory Banks. Current documentation. [Open source](#)
Directives, mental models and transfer behavior; verify against a pinned implementation.